Epistemic Debt in LLM-Augmented Software Engineering

Mitigation Strategies Through TDD, DDD, and Advanced Practices

Research Compilation - January 2026

Executive Summary 3
1. Understanding Epistemic Debt 3
  1.1 Definition and Theoretical Framework 3
  1.2 Three Levels of Epistemic Debt 3
  1.3 Taxonomy of AI-Induced Architectural Erosion 4
2. Test-Driven Development as Mitigation 6
  2.1 Academic Evidence for TDD with LLMs 6
  2.2 Practical Application of TDD 6
  2.3 Integration Testing Specifically 6
  2.4 Key Research Findings 6
3. Domain-Driven Design as Mitigation 8
  3.1 Ubiquitous Language 8
  3.2 Bounded Contexts 8
  3.3 Practical Application with AI 9
4. The Speed vs. Ownership Trade-off 10
  4.1 Deliberate Decision-Making 10
  4.2 Epistemic Debt as a Conscious Choice 10
  4.3 Strategic Application Based on Domain Importance 10
5. Additional Mitigation Strategies 12
  5.1 Human-in-the-Loop Code Review Processes 12
  5.2 Property-Based Testing 12
  5.3 Static Analysis and Multi-Agent Security Frameworks 13
  5.4 Documentation and Knowledge Preservation 13
  5.5 PDCA (Plan-Do-Check-Act) Framework 13
  5.6 Scope Management and Iterative Development 14
  5.7 Multi-Model Review Strategies 14
  5.8 Pair Programming with Clear Role Definitions 14

5.9 Comprehension Debt Management 14
5.10 Software Intelligence and Continuous Monitoring 15
5.11 Deterministic Gates and Defense Mechanisms 15
5.12 Security-Specific Practices 15
6. Practical Guardrails from Industry Practice 16
7. Summary: Mitigation Strategies Overview 17
8. Conclusions and Recommendations 19
8.1 Recommendations for Practice 19
8.2 Future Research Directions 20
9. Key References and Resources 21
9.1 Epistemic Debt Theory 21
9.2 Test-Driven Development with LLMs 21
9.3 Domain-Driven Design and LLMs 21
9.4 Practical Implementation Guidance 21
9.5 Code Review and Human Oversight 22
9.6 Advanced Testing Strategies 22
9.7 Additional Resources 22

Executive Summary

This document presents comprehensive research on epistemic debt in LLM-augmented software engineering, examining the phenomenon where developers can produce systems that exceed their own understanding. Drawing from recent academic papers, industry practices, and expert analysis, this research explores:

- The theoretical framework and formal definition of epistemic debt
- How Test-Driven Development (TDD) mitigates epistemic accumulation
- The role of Domain-Driven Design (DDD) in maintaining understanding
- The deliberate trade-off between speed and epistemic ownership
- Additional mitigation strategies beyond TDD and DDD

1. Understanding Epistemic Debt

1.1 Definition and Theoretical Framework

Epistemic debt represents the divergence between the complexity of a software system and the developer’s cognitive model of that system. As defined by researcher Ludovic Ngabang in a recent paper, unlike traditional technical debt which is often a conscious trade-off, epistemic debt is an invisible accumulation of “unearned” code that functions correctly but lacks a human owner who understands its causality.

The formal mathematical definition: $De = \int(Cs(t) - Gc(t))dt$

Where:

- De = Epistemic Debt
- Cs = System Complexity
- Gc = Cognitive Grasp of the maintenance team

Source: [Defining “Epistemic Debt” in the Era of LLM-Assisted Software Engineering]

In traditional development, Cs and Gc rise in tandem. In AI-assisted development, Cs can rise exponentially while Gc remains flat. When De exceeds a critical threshold, the system becomes “Legacy Code” the moment it is written—unmaintainable and terrifying to touch.

1.2 Three Levels of Epistemic Debt

According to research published in Medium, epistemic debt manifests at three distinct levels:

Level 1: Implementation Mystery

The code works, but the reasoning behind it is a black box. This is common in “vibe coding,” where a developer simply describes the desired outcome and lets the AI figure out the implementation. The risk here is that the AI’s solution might have edge cases or performance characteristics that you are not aware of.

Level 2: Behavioral Uncertainty

Understanding what the code does, but not why it was implemented in that particular way or what assumptions it makes.

Level 3: Architectural Mystery

The most perilous level of not knowing. The AI generates entire modules or services, and you don’t understand how they fit into the larger system. You might have a collection of well-functioning black boxes, but you lack the holistic, systems-level understanding to reason about their interactions. This is where catastrophic failures and security vulnerabilities are born.

Source: The Epistemology Crisis in AI-Assisted Development

1.3 Taxonomy of AI-Induced Architectural Erosion

Research identifies three distinct phenomena that characterize epistemic debt accumulation:

1. The Stochastic Spaghetti Effect

Human architects tend to think in patterns and modular structures. LLMs, being probabilistic token predictors, optimize for local probability rather than global coherence. The model solves Problem A and Problem B in the same file using two completely different coding styles or abstraction levels, simply because the prompts were different. This leads to a breakdown of “Conceptual Integrity,” which Fred Brooks identified as the most important consideration in system design.

2. The Mirage of Correctness (Dependency Hallucination)

LLMs often invent APIs that should exist but do not. This forces the developer to write “shim” code or wrappers to make reality match the hallucination. This reverses the dependency flow—instead of the tool serving the architecture, the architecture is warped to accommodate the stochastic output of the tool.

3. Context Window Amnesia

Even with large context windows (128k+ tokens), LLMs struggle with the “unknown unknowns” of a large codebase. They cannot reason about the side effects a generated function might have on a distant module not included in the prompt context. This results in the introduction of

“Heisenbugs”—bugs that disappear or alter behavior when one attempts to probe them, often caused by subtle state mismanagement generated by the AI.

2. Test-Driven Development as Mitigation

2.1 Academic Evidence for TDD with LLMs

Research demonstrates that providing LLMs like GPT-4 and Llama 3 with tests in addition to problem statements enhances code generation outcomes, enabling developers to verify correctness of generated code against predefined tests. This approach applies TDD principles to LLM-based code generation, offering one distinct benefit: it enables developers to verify the correctness of generated code against predefined tests.

Sources: Test-Driven Development for Code Generation (arXiv) | IEEE Publication

2.2 Practical Application of TDD

When you write tests first, especially including edge cases and boundary conditions, your generated code will try to fulfill those requirements from the beginning, helping avoid nasty surprises down the road. The tests that you have implemented are valuable context for the LLM that will be generating your code, so include them in the conversation or have your IDE-integrated assistant examine the test files.

Source: How AI Code Assistants Are Revolutionizing Test-Driven Development

The TDD approach, dubbed “Test-driven Brute-force-based LLM-assisted Development” (T.B.L.D.), involves defining desired behavior with tests and using LLMs to generate code that passes those tests. Recognizing that LLM-generated code is far from perfect, developers feed test and compilation errors back into the LLM to generate new code, iterating until the code compiles and passes all tests.

Source: Harnessing LLMs with TDD

2.3 Integration Testing Specifically

Integration testing involves API tests to verify that LLMs and backend systems are communicating effectively, integration of semantic search to examine how the LLM works with other systems, and error handling to simulate API failures and verify fallback mechanisms. By thoroughly testing these components, teams can increase AI performance, optimize system interactions, and identify compatibility issues early.

Source: Pre-Deployment Testing for LLM Applications

2.4 Key Research Findings

Experimental evaluations conducted on established function-level code generation benchmarks such as MBPP and HumanEval consistently demonstrate that including test cases leads to higher success in solving programming challenges. The research

asserts that TDD is a promising paradigm for helping ensure that the code generated by LLMs effectively captures the requirements.

A novel benchmark called WebApp1K was introduced for evaluating LLMs in test-driven development tasks, where test cases serve as both prompt and verification for code generation. The findings highlight instruction following and in-context learning as critical capabilities for TDD success, surpassing the importance of general coding proficiency or pretraining knowledge.

Source: Tests as Prompt: A Test-Driven-Development Benchmark for LLM Code Generation

3. Domain-Driven Design as Mitigation

3.1 Ubiquitous Language

Eric Evans, the creator of DDD, recently weighed in on LLMs at the Explore DDD conference. He said a trained language model is a bounded context, and instead of using models trained on broad language intended for general purposes like ChatGPT, training a language model on a ubiquitous language of a bounded context makes it far more effective.

Source: Eric Evans Encourages DDD Practitioners to Experiment with LLMs

Ubiquitous Language is the term Eric Evans uses in Domain Driven Design for the practice of building up a common, rigorous language between developers and users. This language should be based on the Domain Model used in the software—hence the need for it to be rigorous, since software doesn’t cope well with ambiguity.

Source: Martin Fowler - Ubiquitous Language

In the fusion of DDD and AI, the ubiquitous language stands as the bedrock of mutual understanding, ensuring that AI is not a mysterious black box but an integral, comprehensible part of the domain. The ubiquitous language provides the right context for LLMs, helping to constrain outputs and reduce hallucinations. Domain experts can design better prompts when they know what they want and use the correct terminology.

Sources: AI/ML & Domain Driven Design | Domain Driven Design: Tailoring intelligent LLM

3.2 Bounded Contexts

A bounded context, a fundamental concept in Domain-Driven Design, defines a specific area within which a domain model is consistent and valid, ensuring clarity and separation of concerns. In microservices architecture, a bounded context often maps to a microservice.

Source: Domain-Driven Design - Wikipedia

Bounded contexts in DDD encapsulate specific areas where terms and concepts retain explicit and consistent meanings. Introducing AI/ML into this matrix accentuates the need for this strategic compass. When LLMs are trained or prompted with domain-specific language within clear bounded contexts, they produce more accurate and contextually appropriate code.

Source: AI/ML & Domain Driven Design

3.3 Practical Application with AI

AI-powered DDD modeling tools like Qlerify are emerging to help teams visually collaborate, define bounded contexts, and generate API-ready models from simple text prompts. The use of correct ubiquitous language can lead to better and more accurate answers from LLMs. Domain experts know to ask the right questions, and LLMs have likely read extensive information about many domains during training. When domain experts design prompts using the ubiquitous language, the combination becomes powerful.

Sources: Qlerify: The AI-Powered DDD Modeling Tool | Enhancing DDD with Generative AI

4. The Speed vs. Ownership Trade-off

4.1 Deliberate Decision-Making

AI makes exploration cheap and parallel. If you don’t do the normal engineering work—what’s changing, why, what might break, how you’ll know, and how you’ll roll it out—the cost shows up later. This is the fundamental trade-off in LLM-augmented development: speed now versus understanding (and maintainability) later.

Source: Autocomplete is not the point: Agentic AI Coding

4.2 Epistemic Debt as a Conscious Choice

Perhaps epistemic debt is a necessary trade-off for the incredible gains in productivity that AI tools provide. If we are to accept this new reality, we must do so consciously. We need a framework for thinking about and managing epistemological debt—knowing when a lack of understanding is acceptable and when it is a critical risk.

Source: The Epistemology Crisis in AI-Assisted Development

4.3 Strategic Application Based on Domain Importance

Just as DDD distinguishes between core, supporting, and generic subdomains, teams can make deliberate choices about where to accept epistemic debt:

Core Domains

Require deep understanding, minimal epistemic debt. Invest heavily in TDD, integration testing, and maintaining ubiquitous language. These are the domains where competitive advantage lies, and where errors have the highest business impact.

Supporting Domains

Moderate epistemic debt acceptable with guardrails. Apply selective testing and review practices. These domains support the core but aren’t differentiators.

Generic Domains

Higher epistemic debt tolerance—speed over deep understanding may be appropriate. Use standard solutions, possibly with heavier LLM assistance. Focus effort where it matters most.

This strategic approach allows teams to consciously trade speed for understanding in low-stakes areas while maintaining rigorous practices in critical domains, similar to how technical debt can be deliberately chosen in non-core areas.

5. Additional Mitigation Strategies

Beyond TDD and DDD, research has identified numerous additional strategies for managing epistemic debt in LLM-augmented development:

5.1 Human-in-the-Loop Code Review Processes

Structured Review Frameworks

A proposed “Human-in-the-loop LLM Code Review” process ensures accountability where the author, not the LLM, implements suggested changes. A “Review Responsible” determines if the LLM’s review is sufficient or if additional human review is necessary based on the change’s criticality or complexity.

Source: Evaluating Large Language Models for Code Review

Review as Human Alignment

Within teams, code review should change to primarily be a human alignment mechanism. The point of review is to align the human who was operating the LLM with team standards on what is appropriate, not just to fix the code. This represents a fundamental shift in how we think about code review in the LLM era.

Source: Code Review as Human Alignment in the Era of LLMs

Explanation-Forcing Reviews

Explaining your code in review reveals your own bugs—not because LLM or person’s feedback is always correct, but because the act of explaining your logic makes gaps visible. This technique leverages the cognitive benefits of articulation to surface misunderstandings.

Source: Using LLMs for Code Review and Testing

5.2 Property-Based Testing

Property-Based Testing (PBT) validates high-level program properties or invariants, which are often simpler to define and verify than exhaustive test oracles. The Property-Generated Solver framework leverages PBT by focusing on verifiable properties and providing targeted feedback derived from minimized failing inputs.

Source: Use Property-Based Testing to Bridge LLM Code Generation and Validation

Intent-Based Testing using Property-Based Testing verifies invariants. Since we cannot trust the implementation path chosen by the AI, we must rigorously constrain the

boundaries of its behavior. This approach is particularly valuable when dealing with complex logic where exhaustive test cases are impractical.

Source: Defining “Epistemic Debt” in the Era of LLM-Assisted Software Engineering

5.3 Static Analysis and Multi-Agent Security Frameworks

AutoSafeCoder is a multi-agent framework for securing LLM code generation through static analysis and fuzz testing. This approach combines multiple verification techniques to catch security vulnerabilities and correctness issues that might slip through traditional testing.

Source: Awesome Code LLM Repository

5.4 Documentation and Knowledge Preservation

Inverse Documentation Principle

Do not commit AI code until you can explain it to the AI. Developers should force the LLM to generate code, then the developer must write the documentation manually. If the developer cannot document the logic clearly, the code is rejected. This forces the cognitive synchronization required to lower epistemic debt.

Source: Defining “Epistemic Debt” in the Era of LLM-Assisted Software Engineering

LLM-Optimized Documentation

LLM-optimized documentation ensures that AI systems can retrieve and provide accurate, contextual responses. When documentation is structured with predictable patterns such as consistent headings, code annotations, and explicit parameter descriptions, LLMs are more likely to surface correct results.

Source: Optimizing Technical Documentations for LLMs

5.5 PDCA (Plan-Do-Check-Act) Framework

A human-supervised PDCA framework for sustainable development includes:

- Defining and enforcing an explicit “definition of done” that includes delivering the planned output to your personal and team quality standards
- Committing to “one focused change at a time, one failing test at a time, no exceptions”
- Respecting existing architecture by working within established patterns

Source: Reducing AI Code Debt: A Human-Supervised PDCA Framework

5.6 Scope Management and Iterative Development

Break Work into Manageable Chunks

Scope management is everything—feed the LLM manageable tasks, not the whole codebase at once. LLMs do best when given focused prompts: implement one function, fix one bug, add one feature at a time.

Source: My LLM Coding Workflow Going into 2026

Planning Before Coding

Having a clear spec and plan means when we unleash the codegen, both the human and the LLM know exactly what we’re building and why, forcing you and the AI onto the same page and preventing wasted cycles.

5.7 Multi-Model Review Strategies

Multi-model reviews run code through different LLMs—for example, Claude for generation and a security-focused model for audit—to catch biases. This approach recognizes that different models have different strengths and blind spots.

Source: Code Review in the Age of AI

5.8 Pair Programming with Clear Role Definitions

Research found that AI agents successfully facilitated knowledge transfer while failing at providing logical explanations or discussions, suggesting that human-AI pair programming requires different expectations than human-human pairing.

Source: LLM, Your pAIr Programmer?

Maintaining Senior-Junior Dynamic

The LLM is an assistant, not an autonomously reliable coder. Developers must position themselves as the senior developer; the LLM is there to accelerate them, not replace their judgment.

5.9 Comprehension Debt Management

When teams produce code faster than they can understand it, it creates “comprehension debt.” Teams that care about quality will take the time to review and understand (and more often than not, rework) LLM-generated code before it makes it into the repository. This slows things down, but ensures that downstream modifications don’t become nightmare scenarios.

Source: Comprehension Debt: The Ticking Time Bomb of LLM-Generated Code

5.10 Software Intelligence and Continuous Monitoring

Applying software intelligence for continuous oversight through leveraging software mapping and intelligence technologies can accurately understand the quality and potential technical debt of both AI-generated and manually written code.

Source: Artificial Intelligence and Technical Debt

AI tools can go beyond surfacing code smells to creating entire dashboards with hotspots, churn rates, entropy metrics, and even predicted cost-of-change per module, making technical debt legible to the business.

Source: Can AI Solve Your Technical Debt Problem?

5.11 Deterministic Gates and Defense Mechanisms

Defense mechanisms include deterministic gates, human oversight at decision points, tests that fail hard, and no autonomous loops. These create checkpoints where automated systems must prove correctness before proceeding.

Source: Using LLMs for Code Review and Testing

5.12 Security-Specific Practices

If code touches authentication, payments, secrets, or untrusted input, treat AI as a high-speed intern and require a human threat model review plus a security tool pass before merge. Approximately 45% of AI-generated code contains security flaws, making this non-negotiable.

Source: Code Review in the Age of AI

6. Practical Guardrails from Industry Practice

Based on real-world application of these principles, several practical guardrails have emerged:

No Explanation, No Merge

If the author can’t explain what changed, why it changed, what it assumes, and how it can fail, it shouldn’t ship yet. That’s the simplest way to avoid epistemic debt.

Write the Intent Down Before Improving the Code

For any non-trivial change, create a tiny written spec including what behavior is changing, what is explicitly not changing, what assumptions this relies on, and how it was validated. If the model drafts a neat explanation, it’s worth rewriting it in your own words. If you can’t, that’s a signal the change isn’t understood yet.

Keep Diffs Reviewable

Large diffs collapse multiple decisions into one review moment. If something small turns into something big, it’s usually a sign to split it: isolate refactors from behavior changes, and isolate scaffolding from logic.

Treat Tests as Product Code

AI generates plausible UI tests from “what the page looks like today,” and they pass once before turning brittle. Aim for tests that assert intent (what must be true), not incidental structure.

If It Feels Too Easy, Pause

Switching models, changing behavior (not just refactoring), touching multiple concerns, or widening scope should trigger a deliberate check-in. If everything feels effortless, it’s worth assuming something important is being skipped and slowing down on purpose.

Source: Autocomplete is not the point: Agentic AI Coding

7. Summary: Mitigation Strategies Overview

The following table summarizes all identified mitigation strategies, their primary benefits, and recommended application scenarios:

Strategy: Test-Driven Development (especially integration testing)

- Primary Benefit: Verifies correctness against predefined tests
- When to Apply: All LLM-generated code

Strategy: Domain-Driven Design (ubiquitous language & bounded contexts)

- Primary Benefit: Aligns AI output with business domain
- When to Apply: Core domains, complex systems

Strategy: Human-in-the-loop code review

- Primary Benefit: Ensures human understanding and alignment
- When to Apply: Critical/complex changes

Strategy: Property-based testing

- Primary Benefit: Validates invariants without exhaustive oracles
- When to Apply: Complex logic, edge cases

Strategy: Static analysis + fuzz testing

- Primary Benefit: Security and correctness verification
- When to Apply: Security-sensitive code

Strategy: Inverse documentation

- Primary Benefit: Forces developer understanding
- When to Apply: Non-trivial changes

Strategy: PDCA framework

- Primary Benefit: Systematic quality control
- When to Apply: Team-wide standards

Strategy: Scope management

- Primary Benefit: Reduces complexity per task
- When to Apply: All AI-assisted development

Strategy: Multi-model review

- Primary Benefit: Catches model-specific biases
- When to Apply: High-stakes code

Strategy: Comprehension debt tracking

- Primary Benefit: Prevents knowledge loss

- When to Apply: Ongoing maintenance

Strategy: Software intelligence monitoring

- Primary Benefit: Continuous quality oversight
- When to Apply: Enterprise systems

Strategy: Deterministic gates

- Primary Benefit: Prevents autonomous errors
- When to Apply: Production systems

8. Conclusions and Recommendations

The research on epistemic debt in LLM-augmented software engineering reveals a complex landscape where speed and understanding exist in tension. Several key conclusions emerge:

1. Epistemic Debt is Real and Measurable

The formal definition $De = \int(Cs(t) - Gc(t))dt$ provides a framework for understanding and discussing this phenomenon. It’s not merely a theoretical concern but a practical challenge that manifests in three distinct levels: implementation mystery, behavioral uncertainty, and architectural mystery.

2. Traditional Software Engineering Practices Remain Essential

TDD and DDD are not rendered obsolete by AI—they become more important. These practices provide the structure and discipline necessary to maintain understanding in an environment where code generation has become nearly effortless.

3. Strategic Trade-offs Are Necessary

Just as technical debt can be deliberately chosen, epistemic debt can be strategically accepted in non-core domains. The key is making this choice consciously and applying different levels of rigor based on domain importance.

4. Layered Defense is Required

No single mitigation strategy is sufficient. The most effective approach combines multiple complementary techniques: TDD for correctness, DDD for alignment, human-in-the-loop review for understanding, property-based testing for invariants, and continuous monitoring for quality.

5. Human Judgment Remains Central

The role of the developer shifts from code author to code curator, but this doesn’t diminish their importance. If anything, the judgment required to maintain system coherence, architectural integrity, and team alignment becomes more critical.

8.1 Recommendations for Practice

For Individual Developers:

- Never merge code you can’t explain
- Write tests before or alongside AI code generation
- Document your understanding, not just the code
- Treat LLMs as junior colleagues: fast but requiring oversight

For Teams:

- Establish explicit guardrails for AI-generated code

- Make AI usage visible in code reviews
- Invest in ubiquitous language and bounded context definition
- Conduct regular epistemic debt audits

For Organizations:

- Classify domains by importance and apply appropriate rigor
- Invest in developer education on epistemic debt management
- Implement software intelligence monitoring for AI-generated code
- Balance speed metrics with understanding metrics

8.2 Future Research Directions

Several areas warrant further investigation:

- Quantitative metrics for measuring epistemic debt in real codebases
- Long-term studies on the maintainability of AI-generated code
- Automated tools for detecting epistemic debt accumulation
- Best practices for training LLMs on domain-specific ubiquitous language
- The relationship between epistemic debt and traditional technical debt

9. Key References and Resources

This research compilation draws from numerous academic papers, industry articles, and expert analyses. The most significant sources are listed below, organized by topic area.

9.1 Epistemic Debt Theory

Defining “Epistemic Debt” in the Era of LLM-Assisted Software Engineering - Ludovic Ngabang (2026)

The foundational academic paper that formally defines epistemic debt and provides the mathematical framework.

The Epistemology Crisis in AI-Assisted Development

Practical exploration of how epistemic debt manifests in real development scenarios.

9.2 Test-Driven Development with LLMs

Test-Driven Development for Code Generation

Academic research demonstrating that providing tests alongside problem statements enhances LLM code generation outcomes.

Tests as Prompt: A Test-Driven-Development Benchmark for LLM Code Generation

WebApp1K benchmark showing that instruction following and in-context learning are critical for TDD success.

How AI Code Assistants Are Revolutionizing Test-Driven Development

Practical guide to implementing TDD with AI coding assistants.

9.3 Domain-Driven Design and LLMs

Eric Evans Encourages DDD Practitioners to Experiment with LLMs

DDD creator Eric Evans on training LLMs with ubiquitous language within bounded contexts.

AI/ML & Domain Driven Design

How ubiquitous language and bounded contexts apply to AI/ML integration.

9.4 Practical Implementation Guidance

Autocomplete is not the point: Agentic AI Coding

Comprehensive practical guardrails for managing epistemic debt in day-to-day development.

My LLM Coding Workflow Going into 2026

Detailed workflow for maintaining understanding while using LLM assistance.

Reducing AI Code Debt: A Human-Supervised PDCA Framework

Systematic framework for sustainable AI-assisted development.

9.5 Code Review and Human Oversight

Evaluating Large Language Models for Code Review

Research on human-in-the-loop code review processes.

Code Review as Human Alignment in the Era of LLMs

Rethinking code review as an alignment mechanism rather than just error detection.

Code Review in the Age of AI

Multi-model review strategies and security considerations.

9.6 Advanced Testing Strategies

Use Property-Based Testing to Bridge LLM Code Generation and Validation

Property-Based Testing framework for validating LLM-generated code.

Pre-Deployment Testing for LLM Applications

Comprehensive guide to integration testing for LLM-based systems.

9.7 Additional Resources

Awesome Code LLM Repository

Curated collection of research papers on code LLMs, including security frameworks.

Artificial Intelligence and Technical Debt: Navigating the New Frontier

Software intelligence approaches to monitoring AI-generated code quality.

Comprehension Debt: The Ticking Time Bomb of LLM-Generated Code

Discussion of the downstream costs of producing code faster than it can be understood.

End of Research Document

Compiled January 2026